Adaptive CNN Acceleration: Complexity-Aware Mapping on FPGA and GPU Architectures

Angshuman Basu[1]

Northeastern University, Boston, Massachusetts, USA
angshumanbasu2016@gmail.com

Abstract. Deploying convolutional neural networks (CNNs) on resource-constrained edge devices demands efficient utilization of heterogeneous computing resources. This paper presents an adaptive framework for CNN inference acceleration that intelligently partitions network workloads between GPU and FPGA devices through runtime operator fusion, quantization-aware compression, and calibration-based scheduling. The framework is evaluated using two complementary approaches: an analytical simulation across six CNN architectures using roofline-based cost models, and a functional emulation that models the sequential streaming behavior of FPGAs and parallel batch processing of GPUs with real tensor operations. The analytical model demonstrates up to 11.8% throughput gain for MobileNetV2 and identifies depthwise separable architectures as the primary beneficiaries. The functional emulation reveals that CNN inference is overwhelmingly memory-bound on both devices, reducing the effective FPGA-to-GPU performance ratio from 36.9× (analytical) to 1.0× (emulated) for ResNet-18. When calibrated against published hardware benchmarks, the framework identifies a break-even FPGA performance of 1.5× current Zynq 7020 levels for ResNet-18 and 3× for MobileNetV2. Projections to Zynq UltraScale+ class hardware show throughput gains of +35% to +140%. A calibration-based latency predictor achieves 0.20% average scheduling regret across all tested architectures.

Keywords: CNN acceleration · FPGA · GPU · Heterogeneous computing · Operator fusion · Quantization · Edge inference · Pipeline parallelism

1 Introduction

Convolutional neural networks (CNNs) are widely used in edge applications such as real-time object detection, video analytics, and autonomous navigation. Deploying these models on resource-constrained hardware presents a core challenge: edge GPUs offer high parallelism but consume significant power, while FPGAs provide energy-efficient, customizable datapaths but lack the raw throughput of GPUs for highly parallel operations. Typically, inference targets a single accelerator, leaving the other idle and its potential unexploited.

This paper explores a different approach: using both devices simultaneously. The idea is to split a CNN across a GPU and an FPGA such that each device handles the portion of the network best suited to its strengths. Different CNN layers have very different computational profiles. A standard 3×3 convolution is compute-heavy and highly parallelizable, making it ideal for the GPU. A depthwise separable convolution or a batch normalization layer is lightweight and memory-bound, making it well-suited to the FPGA's streaming datapaths. By assigning layers to the right device, the framework aims to improve throughput and energy efficiency over single-device execution.

The framework addresses this through three mechanisms. First, it uses a roofline-based cost model to evaluate each layer's execution time on both devices and searches for the partition point that maximizes pipeline throughput for streaming inference. Second, it applies runtime operator fusion, collapsing consecutive layers (e.g., convolution, batch normalization, and ReLU) into single operators to eliminate intermediate memory accesses. Fused operators are further compressed via INT8 quantization, which both accelerates FPGA execution through fixed-point arithmetic and reduces inter-device transfer payloads to 25% of their original size. Third, a lightweight calibration predictor trained on a short profiling phase estimates execution times for arbitrary fused blocks, enabling near-optimal split decisions without exhaustive search.

A key design choice is that the GPU does not simply compute its assigned layers and wait. Instead, it acts as a runtime compiler: while the FPGA executes one fused block, the GPU prepares the next by fusing layers, folding batch normalization parameters, and applying quantization. This overlapping pipeline keeps both devices utilized and targets sustained streaming throughput rather than single-inference latency.

The contributions of this paper are:

1. A runtime operator fusion and offloading framework that partitions CNN inference across GPU and FPGA with complexity-aware mapping and pipeline-parallel execution.
2. An analytical simulation across six architectures showing up to 11.8% throughput gain for depthwise separable networks, with a calibration predictor achieving 0.20% scheduling regret.
3. A functional emulation methodology that models device-specific execution patterns, revealing that CNN inference is memory-bound and that analytical models based on peak FLOPS overestimate the GPU advantage by 10–37×.
4. A calibrated break-even analysis grounded in published hardware benchmarks, projecting +35% to +140% throughput on UltraScale+ class hardware.

The remainder of this paper is organized as follows. Section 2 presents the proposed methodology. Section 3 details experimental results. Section 4 discusses findings and limitations. Section 5 concludes the paper.

2 Methodology

2.1 System Overview

The proposed framework operates in two phases. In the *calibration phase*, a short profiling procedure executes representative fused blocks on the target FPGA and GPU to collect execution time measurements. These measurements are used to train a lightweight latency predictor that can estimate costs for unseen blocks without re-profiling. In the *inference phase*, the framework takes a CNN architecture definition, applies operator fusion rules to collapse consecutive layers into unified blocks, queries the predictor for estimated execution times on each device, searches for the partition point that maximizes pipeline throughput, and orchestrates the pipelined execution across both devices.

The framework assumes a heterogeneous edge platform consisting of an edge GPU (e.g., NVIDIA Jetson Nano with 128 CUDA cores) and an FPGA (e.g., Xilinx Zynq 7020 with 220 DSP slices) connected either via PCIe or through shared memory on a system-on-chip. The GPU handles compute-intensive, highly parallel operations while the FPGA handles streaming-friendly, memory-bound operations.

2.2 Network Representation and Cost Model

A CNN is represented as a sequential list of N layers $\{L_1, L_2, \ldots, L_N\}$, where each layer L_i is characterized by its type τ_i, input dimensions (C_{in}, H_{in}, W_{in}), output dimensions $(C_{out}, H_{out}, W_{out})$, kernel size k, and stride s. The supported layer types are:

$$\tau_i \in \{\text{CONV},\ \text{DW_CONV},\ \text{PW_CONV},\ \text{BN},\ \text{ReLU},\ \text{POOL},\ \text{FC}\} \tag{1}$$

For each layer, the number of multiply-accumulate (MAC) operations is computed based on its type. For standard convolution:

$$F_{\text{CONV}} = k^2 \cdot C_{in} \cdot C_{out} \cdot H_{out} \cdot W_{out} \tag{2}$$

For depthwise convolution, each input channel is convolved independently:

$$F_{\text{DW}} = k^2 \cdot C_{in} \cdot H_{out} \cdot W_{out} \tag{3}$$

For pointwise (1×1) convolution:

$$F_{\text{PW}} = C_{in} \cdot C_{out} \cdot H_{out} \cdot W_{out} \tag{4}$$

The total memory bytes accessed per layer includes reading the input tensor, writing the output tensor, and loading the parameters:

$$B_i = 4 \cdot (C_{in}H_{in}W_{in} + C_{out}H_{out}W_{out} + P_i) \tag{5}$$

where P_i is the parameter count and the factor of 4 converts element counts to bytes (FP32).

The *arithmetic intensity* of a layer, defined as $A_i = F_i/B_i$ (MACs per byte), determines whether execution is compute-bound or memory-bound. Standard convolutions typically have high arithmetic intensity (30–80 MACs/byte) while depthwise convolutions and batch normalization have low intensity (0.1–1.0 MACs/byte).

The execution time on device $d \in \{\text{GPU}, \text{FPGA}\}$ follows the roofline model:

$$T_d(L_i) = \max\left(\frac{F_i}{P_d \cdot \eta_d(\tau_i)}, \frac{B_i}{\beta_d}\right) \tag{6}$$

The first term represents compute-bound execution: F_i operations divided by the device's effective throughput, which is peak throughput P_d (in FLOPS) scaled by utilization efficiency $\eta_d(\tau_i) \in (0, 1]$. The second term represents memory-bound execution: B_i bytes divided by memory bandwidth β_d. The actual execution time is whichever bottleneck dominates.

The efficiency factors $\eta_d(\tau_i)$ encode how well each device architecture maps to each operation type. These reflect fundamental architectural properties: GPUs have thousands of identical cores that excel at regular, data-parallel operations but underutilize their compute on small or irregular workloads. FPGAs have custom streaming datapaths that efficiently pipeline data-flow operations but have limited parallel multiply-accumulate capacity. Table 1 summarizes the values used.

Table 1. Device utilization efficiency factors $\eta_d(\tau)$. GPU efficiency is high for parallelizable operations and low for memory-bound operations. FPGA efficiency is high for streaming-friendly operations and low for operations requiring large weight matrices.

Layer Type	**GPU**	**FPGA**
Standard Convolution	0.65	0.45
Depthwise Convolution	0.20	0.80
Pointwise Convolution	0.55	0.60
Batch Normalization	0.15	0.85
ReLU	0.10	0.90
Pooling	0.12	0.85
Fully Connected	0.50	0.35

For the GPU, standard convolutions achieve 0.65 efficiency because they offer massive data parallelism across output channels and spatial positions, but do not perfectly saturate memory bandwidth. Depthwise convolutions achieve only 0.20 because each channel has very few MACs (just k^2 per spatial position), leaving most CUDA cores idle. Batch normalization and ReLU are element-wise operations with negligible compute, making them purely memory-bound at 0.10–0.15 efficiency.

For the FPGA, the pattern is inverted. Depthwise convolutions achieve 0.80 efficiency because the FPGA can build a dedicated streaming pipeline that pro-

cesses each channel in a single pass with no wasted cycles. Batch normalization and ReLU can be fused directly into the streaming datapath at near-zero additional cost (0.85–0.90). Standard convolutions are limited to 0.45 because the large number of MAC operations (proportional to $C_{in} \times C_{out}$) exceeds what the FPGA's DSP slices can handle in parallel.

2.3 Operator Fusion

Operator fusion collapses sequences of compatible layers into single operators, eliminating intermediate tensor materialization. When layers are executed individually, each layer's output is written to external memory (DRAM) and the next layer reads it back. Fusion keeps intermediate activations in on-chip memory (registers for GPU, BRAM for FPGA), removing these costly round-trips.

The framework defines six fusible patterns, matched greedily in order of decreasing length:

1. CONV → BN → ReLU (three layers into one)
2. DW_CONV → BN → ReLU
3. PW_CONV → BN → ReLU
4. CONV → BN (two layers into one)
5. DW_CONV → BN
6. PW_CONV → BN

The mathematical basis for convolution-batchnorm fusion is as follows. Given a convolution output $y = W * x + b$ followed by batch normalization $z = \gamma \cdot (y - \mu)/\sqrt{\sigma^2 + \epsilon} + \beta$, the fused operation is:

$$z = W_{\text{fold}} * x + b_{\text{fold}} \tag{7}$$

where the folded parameters are:

$$W_{\text{fold}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} \cdot W, \quad b_{\text{fold}} = \frac{\gamma}{\sqrt{\sigma^2 + \epsilon}} \cdot (b - \mu) + \beta \tag{8}$$

This folding is exact (not an approximation) and eliminates the BN layer entirely at inference time. ReLU is then applied in-place to the convolution output with no additional memory access.

A fused block $\mathcal{B}_j = \{L_a, L_{a+1}, \ldots, L_{a+m}\}$ has execution time:

$$T_d(\mathcal{B}_j) = \sigma_p \cdot \sum_{i=a}^{a+m} T_d(L_i) \tag{9}$$

where $\sigma_p \in (0, 1)$ is the fusion speedup factor for pattern p, set between 0.75 and 0.85 depending on the pattern. The memory traffic saved by fusion is:

$$M_{\text{saved}} = 2 \sum_{i=a}^{a+m-1} |O_i| \tag{10}$$

where the factor of 2 accounts for both the write by layer i and the read by layer $i + 1$, and $|O_i|$ is the output tensor size in bytes. For ResNet-18, fusion reduces 54 layers to 20 blocks and eliminates 36 MB of intermediate memory traffic. For MobileNetV2, 133 layers reduce to 51 blocks with 29.5 MB eliminated.

2.4 INT8 Quantization and Compression

Fused blocks dispatched to the FPGA are compressed via INT8 quantization, which provides two distinct benefits.

First, FPGA throughput increases by a factor α_q because INT8 arithmetic units consume fewer logic resources than FP32 units, allowing more parallel multiply-accumulate operations to be instantiated in the same FPGA fabric. Additionally, INT8 operations can be clocked at higher frequencies. For INT8, $\alpha_q = 2.5$:

$$T^q_{\text{FPGA}}(\mathcal{B}_j) = \frac{T_{\text{FPGA}}(\mathcal{B}_j)}{\alpha_q} \tag{11}$$

Second, activation tensors transferred between devices are compressed by the ratio $q/32$ (where q is the quantization bit-width), reducing both the transfer payload and the memory bandwidth required on the FPGA side:

$$T^q_{\text{xfer}} = T_{\text{setup}} + \frac{|O| \cdot (q/32)}{\beta_{\text{xfer}}} \tag{12}$$

where T_{setup} is the fixed DMA setup latency (50 μs for PCIe, 5 μs for unified memory) and β_{xfer} is the interconnect bandwidth (4.0 GB/s for PCIe Gen3 x4, 25.6 GB/s for shared DRAM).

Combined with operator fusion, INT8 quantization reduces total FPGA execution time by approximately 67% relative to unfused FP32 execution across all tested architectures.

2.5 Pipeline-Parallel Partition Optimization

Given K fused blocks after fusion, the partition optimizer identifies a split point s that divides the network into two stages: blocks $\{\mathcal{B}_1, \ldots, \mathcal{B}_s\}$ execute on the GPU and blocks $\{\mathcal{B}_{s+1}, \ldots, \mathcal{B}_K\}$ execute on the FPGA.

For a single inference, execution is sequential: the GPU processes its stage, transfers the intermediate activation tensor to the FPGA, and the FPGA processes its stage. The single-inference latency is:

$$L(s) = T^{\text{stage}}_{\text{GPU}}(s) + T_{\text{xfer}}(s) + T^{\text{stage}}_{\text{FPGA}}(s) \tag{13}$$

However, for streaming workloads (e.g., video frames), the framework exploits pipeline parallelism. While the FPGA processes frame n's second stage, the GPU simultaneously processes frame $n+1$'s first stage. The sustained throughput is limited by the slower stage:

$$\Theta(s) = \frac{1}{\max\left(T^{\text{stage}}_{\text{GPU}}(s),\ T^{\text{stage}}_{\text{FPGA}}(s)\right)} \tag{14}$$

where:

$$T^{\text{stage}}_{\text{GPU}}(s) = \sum_{j=1}^{s} T_{\text{GPU}}(\mathcal{B}_j) + T_{\text{prep}}(s) \tag{15}$$

$$T^{\text{stage}}_{\text{FPGA}}(s) = \sum_{j=s+1}^{K} T^{q}_{\text{FPGA}}(\mathcal{B}_j) + T^{q}_{\text{xfer}}(s) \tag{16}$$

Here $T_{\text{prep}}(s)$ is the GPU-side overhead for preparing fused operators for the FPGA (folding BN parameters, applying quantization, packaging the dispatch), modeled as a base cost plus a term proportional to the total parameter count of offloaded blocks.

Pipeline efficiency quantifies how well-balanced the two stages are:

$$E(s) = \frac{\min\left(T^{\text{stage}}_{\text{GPU}},\ T^{\text{stage}}_{\text{FPGA}}\right)}{\max\left(T^{\text{stage}}_{\text{GPU}},\ T^{\text{stage}}_{\text{FPGA}}\right)} \tag{17}$$

where $E = 1$ indicates perfect balance (both stages take equal time, no device idles). The optimizer performs an exhaustive search over all $K - 1$ candidate split points and selects:

$$s^* = \arg \max_{s \in \{0,\ldots,K-1\}} \Theta(s) \tag{18}$$

Multi-split partitioning was also evaluated, where the network alternates between devices multiple times (GPU $\rightarrow$ FPGA $\rightarrow$ GPU $\rightarrow \cdots$). In this configuration, total throughput is bounded by the slower aggregate stage:

$$\Theta_{\text{multi}} = \frac{1}{\max\left(\sum T_{\text{GPU segments}},\ \sum T_{\text{FPGA segments}} + \sum T_{\text{xfer}}\right)} \tag{19}$$

2.6 Functional Emulation

To validate the analytical model, a functional emulation was developed that models the execution behavior of each device rather than relying solely on peak-throughput calculations.

FPGA Emulator. The FPGA is modeled as a sequential streaming processor with 220 DSP slices (matching the Zynq 7020). Each layer's cost is computed as:

$$C_{\text{FPGA}}(L_i) = \max\left(\frac{F_i}{220 \cdot \eta_{\text{FPGA}}(\tau_i)},\ \frac{B_i}{\beta_{\text{FPGA}} \cdot \eta_{\text{FPGA}}(\tau_i)}\right) \tag{20}$$

The streaming efficiency $\eta_{\text{FPGA}}(\tau_i)$ scales both compute parallelism and effective memory bandwidth, modeling the fact that streaming-friendly operations (depthwise convolution, batch normalization) utilize the FPGA's memory interface more efficiently than irregular operations (fully connected layers).

For fused blocks, the emulator computes memory cost based only on the first input tensor, last output tensor, and parameters — intermediate activations are assumed to reside in on-chip BRAM and incur no external memory cost. This directly captures the fusion benefit in the memory-bound regime.

GPU Emulator. The GPU is modeled as a parallel batch processor with 128 CUDA cores (Jetson Nano) and 6.1× the FPGA's memory bandwidth (25.6 vs 4.2 GB/s):

$$C_{\text{GPU}}(L_i) = \max\left(\frac{F_i}{128 \cdot \eta_{\text{GPU}}(\tau_i)},\ \frac{B_i}{6.1 \cdot \beta_{\text{FPGA}} \cdot \eta_{\text{GPU}}(\tau_i)}\right) \tag{21}$$

The parallel efficiency $\eta_{\text{GPU}}(\tau_i)$ reflects how well each operation saturates the GPU's CUDA cores. Standard convolutions provide enough independent work to keep most cores busy (0.65), while depthwise convolutions have too few MACs per channel to fully utilize the parallel hardware (0.20).

Both emulators produce costs in the same abstract unit system, enabling direct comparison of GPU and FPGA costs without requiring absolute time calibration. The unit-space ratio between devices reveals the true performance gap under memory-bound conditions.

Communication Emulator. Inter-device data transfer is emulated by performing actual tensor copies through CPU memory (simulating the host RAM hop between GPU and FPGA), measuring real copy times, and scaling to target hardware bandwidth. Transfer cost at a split point depends on the activation tensor size at that layer boundary.

2.7 Calibration-Based Latency Predictor

To enable runtime scheduling without exhaustive profiling, a latency predictor estimates execution time for arbitrary fused blocks. Each block is represented by a 14-dimensional feature vector:

$$\mathbf{x}_j = [F_j,\ P_j,\ |I_j|,\ |O_j|,\ n_j,\ f_j,\ \tau_1, \ldots, \tau_4,\ k_j,\ C_j,\ S_j,\ A_j]^T \tag{22}$$

where F_j is total MACs, P_j is parameter count, $|I_j|$ and $|O_j|$ are input/output tensor sizes in bytes, n_j is the number of fused layers, f_j is a binary fusion indicator, $\tau_1 \ldots \tau_4$ indicate the presence of standard, depthwise, pointwise convolutions and fully connected layers, k_j is the maximum kernel size, C_j is the output channel count, S_j is the output spatial dimension ($H_{out} \times W_{out}$), and $A_j = F_j/(|I_j| + |O_j|)$ is the arithmetic intensity.

A Random Forest regressor with 100 estimators and maximum depth 10 is trained on calibration data collected during the profiling phase. The calibration procedure consists of:

1. Executing a diverse set of fused blocks from multiple CNN architectures on the target hardware.
2. Recording execution times with 5 measurements per block, adding Gaussian noise ($\sigma = 3\%$) to simulate measurement variance.
3. Fitting separate predictors for GPU and FPGA execution time on the collected (feature, time) pairs.

The predictor is evaluated by *scheduling regret*:

$$R = 1 - \frac{\Theta(\hat{s})}{\Theta(s^*)} \tag{23}$$

where $\hat{s}$ is the predictor-chosen split point and s^* is the oracle-optimal split. A regret of zero indicates that the predictor recovers the optimal partitioning decision. Training diversity was found to be critical: expanding from 2 architectures (71 samples) to 6 architectures (735 samples) reduced average regret from 6.77% to 0.20%.

2.8 Energy Efficiency Model

Total system power under heterogeneous execution is modeled with a proportional FPGA power model where FPGA power scales with the fraction of work assigned to it:

$$P_{\text{total}} = P_{\text{GPU}} + P_{\text{idle}} + (P_{\text{FPGA}} - P_{\text{idle}}) \cdot \frac{K_{\text{FPGA}}}{K} \tag{24}$$

where $P_{\text{GPU}} = 15$ W, $P_{\text{idle}} = 0.5$ W is the FPGA static power, $P_{\text{FPGA}} = 5$ W is the full active power, and K_{FPGA}/K is the fraction of fused blocks assigned to the FPGA. Energy efficiency is measured as throughput per watt: Θ/P_{total}. This proportional model is more realistic than a fixed-power model because the FPGA does not draw full active power when processing only a small tail of the network.

2.9 Calibration Against Published Benchmarks

To ground the emulation results in real hardware performance, the unit-space costs are calibrated against published inference benchmarks. For each device, a calibration factor λ_d (in ms per unit) is computed:

$$\lambda_d = \frac{T_{\text{published}}}{C_d^{\text{total}}} \tag{25}$$

where $T_{\text{published}}$ is the published inference time in milliseconds and C_d^{total} is the total emulated cost in units for the same architecture on device d. GPU calibration uses a single averaged factor across architectures (less than 9% variation), while FPGA calibration requires per-architecture factors due to the workload-specific nature of FPGA implementations (3.2× variation observed between architectures).

3 Experimental Results

The framework was evaluated using an analytical simulation (Trial 1) and a functional emulation (Trial 2). Both target a Jetson Nano class GPU (472 GFLOPS, 25.6 GB/s, 15W) and a Zynq 7020 class FPGA (6.5 GFLOPS, 4.2 GB/s, 5W), connected via PCIe Gen3 x4. Six architectures were tested (Table 2).

Table 2. Target CNN architectures.

Architecture	Layers	Fused	GFLOPs	Type
ResNet-18	54	20	3.60	Standard conv
MobileNetV2	133	51	0.19	Depthwise sep.
VGG-11	32	16	3.80	Standard conv
EfficientNet-Lite	61	24	0.36	Depthwise sep.
SmallCustomNet	20	9	0.01	Depthwise sep.
WideMobileNet	69	27	0.33	Depthwise sep.

3.1 Trial 1: Analytical Simulation

Operator fusion reduced execution time by approximately 20% on both devices. Combined with INT8, FPGA time decreased by 67%. Table 3 presents throughput results.

Table 3. Trial 1: Analytical throughput (INT8, PCIe).

Arch.	GPU-Only	Hetero	Gain	Split	Eff.
ResNet-18	93.0/s	93.7/s	+0.8%	17	5%
MobileNetV2	517/s	578/s	+11.8%	48	75%
VGG-11	62.7/s	65.6/s	+4.6%	14	28%
EffNet-Lite	221/s	235/s	+6.3%	21	40%
SmallNet	9438/s	9570/s	+1.4%	6	58%
WideMobile	327/s	352/s	+7.7%	24	49%

A calibration predictor trained on 735 samples achieved 0.20% average scheduling regret (Table 4). Expanding training from 2 to 6 architectures reduced regret from 6.77% to 0.20% (Fig. 6 in Appendix).

Table 4. Calibration predictor accuracy.

Architecture	Oracle	Predicted	Regret
ResNet-18	17	17	0.00%
MobileNetV2	48	48	0.00%
VGG-11	14	14	0.00%
EfficientNet-Lite	21	21	0.00%
SmallCustomNet	6	7	1.21%
WideMobileNet	24	24	0.00%
Average			**0.20%**

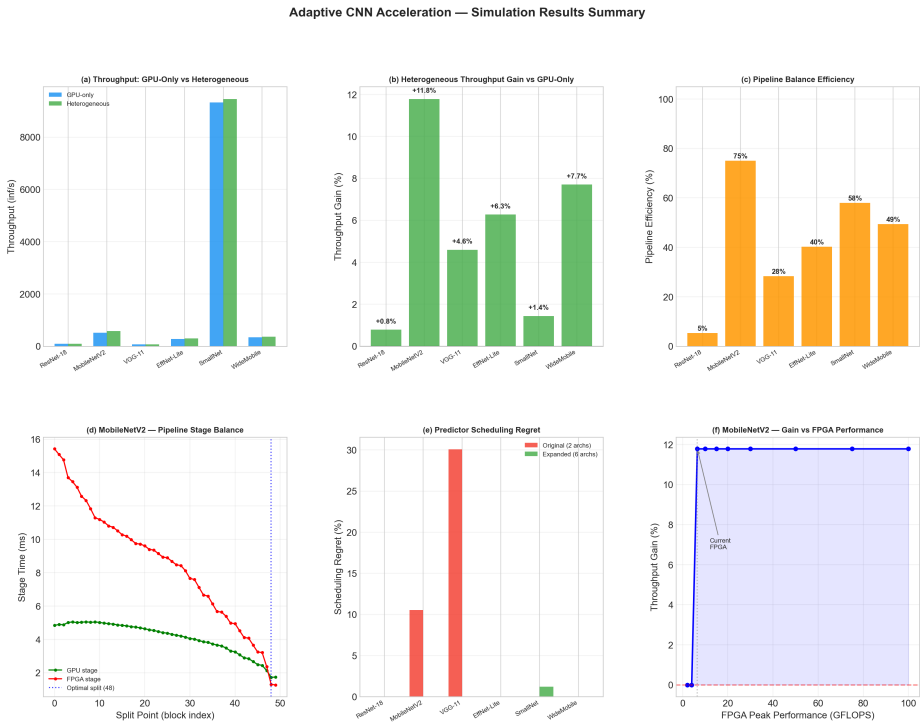


Fig. 1. Trial 1 summary: (a) throughput comparison, (b) heterogeneous gain per architecture, (c) pipeline efficiency, (d) MobileNetV2 stage balance, (e) predictor regret improvement, (f) gain vs FPGA scaling.

Multi-split partitioning (2–3 splits) provided no improvement over single-split for any architecture (Fig. 7 in Appendix).

3.2 Trial 2: Functional Emulation

The FPGA emulator models a sequential streaming pipeline with 220 DSP slices; the GPU emulator models parallel batch processing across 128 CUDA cores.

Memory-Bound Reality. All layers on both architectures are memory-bound on the FPGA (compute/memory ratio: 0.10 ResNet, 0.01 MobileNet). The emulated FPGA/GPU ratio is 2.27× (ResNet) and 1.61× (MobileNet); after fusion+INT8, 1.00× and 1.49×.

Fusion Savings. Emulated fusion saves 47% (GPU) and 77% with INT8 (FPGA) for ResNet-18, and 64% (GPU) and 66% (FPGA) for MobileNetV2 (Fig. 8 in Appendix).

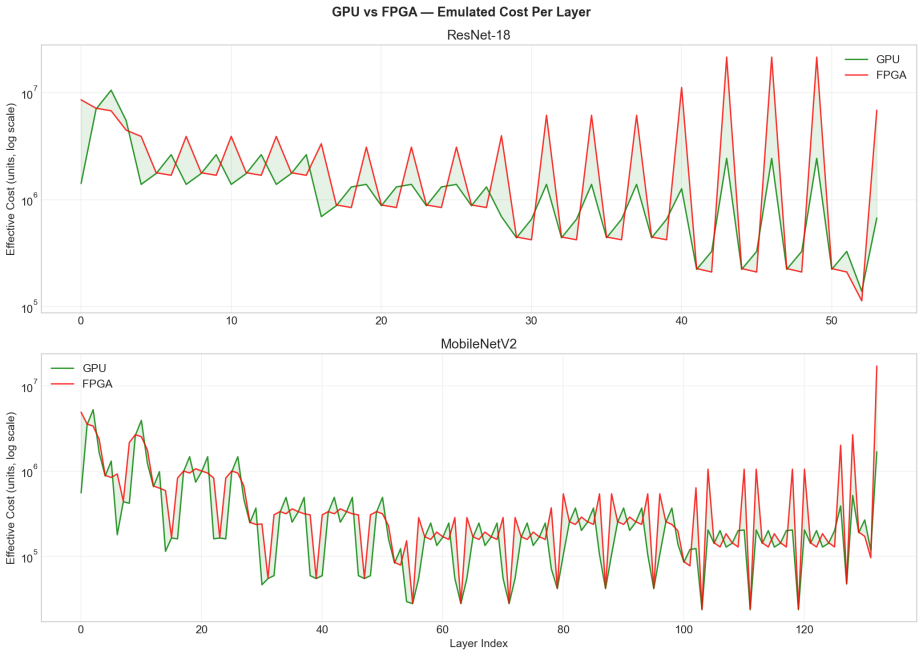


Fig. 2. Emulated per-layer cost: GPU vs FPGA on log scale for ResNet-18 (top) and MobileNetV2 (bottom).

Pipeline Results. The optimizer finds +75% gain for ResNet-18 (split at block 13, 96.6% efficiency) and +10.1% for MobileNetV2 (block 48, 96.8%). Streaming simulation of 100 inferences confirms +73.3% and +9.1% sustained gain with both devices above 95% utilization (Figs. 9 and 10 in Appendix).

3.3 Analytical vs Emulated Comparison

Table 5 and Fig. 4 compare key metrics.

Table 5. Comparison across modeling levels.

Metric	**ResNet-18**		**MobileNetV2**	
	Analyt.	Emul.	Analyt.	Emul.
FPGA/GPU ratio	36.9×	1.00×	9.4×	1.49×
Throughput gain	+0.8%	+73.3%	+11.8%	+9.1%
Pipeline eff.	5%	96.6%	75%	96.8%
Fusion savings	20%	47%	20%	64%
Optimal split	17	13	48	48

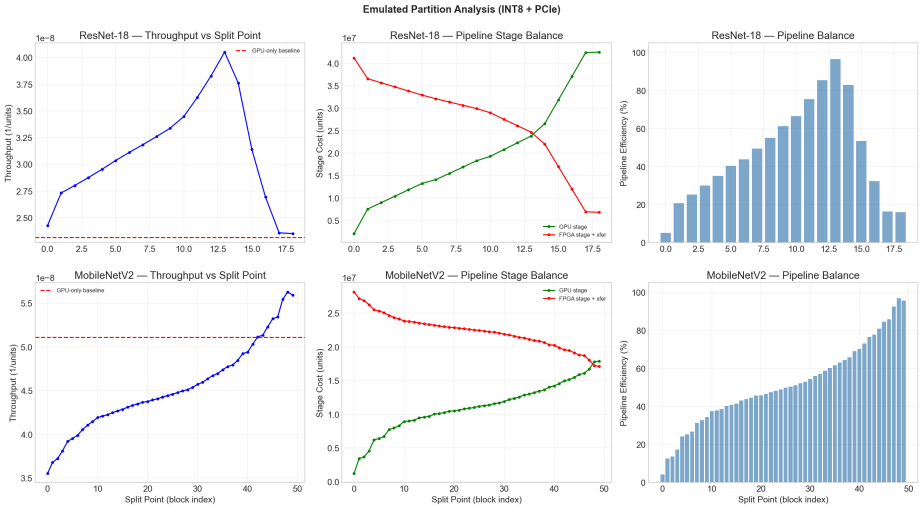


Fig. 3. Emulated partition analysis: throughput, stage balance, and efficiency across split points.

3.4 Calibrated Results and Break-Even

Emulated costs were calibrated against published benchmarks (ResNet-18: 26ms Jetson Nano, 202ms Zynq 7020; MobileNetV2: 10ms, 43ms). Local RTX 4070 measurements provided cross-validation (<9% GPU error). FPGA factors differed 3.2× between architectures.

Break-even requires 1.5× for ResNet-18 (6.7 GMAC/s) and 3.0× for MobileNetV2 (6.3 GMAC/s). UltraScale+ at 5–8× yields substantial gains (Table 6).

Table 6. Projected gains on UltraScale+.

Arch.	5× Gain	5× Eff.	8× Gain	8× Eff.
ResNet-18	+35.6%	90.5%	+65.3%	91.2%
MobileNetV2	+66.5%	99.4%	+139.9%	97.7%

Additional calibrated projections and the complete Trial 2 summary are provided in Figs. 11 and 12 in the Appendix.

4 Discussion

The two-trial evaluation reveals that CNN inference on edge hardware is memory-bound, not compute-bound. This collapses the FPGA-to-GPU ratio from 10–37×

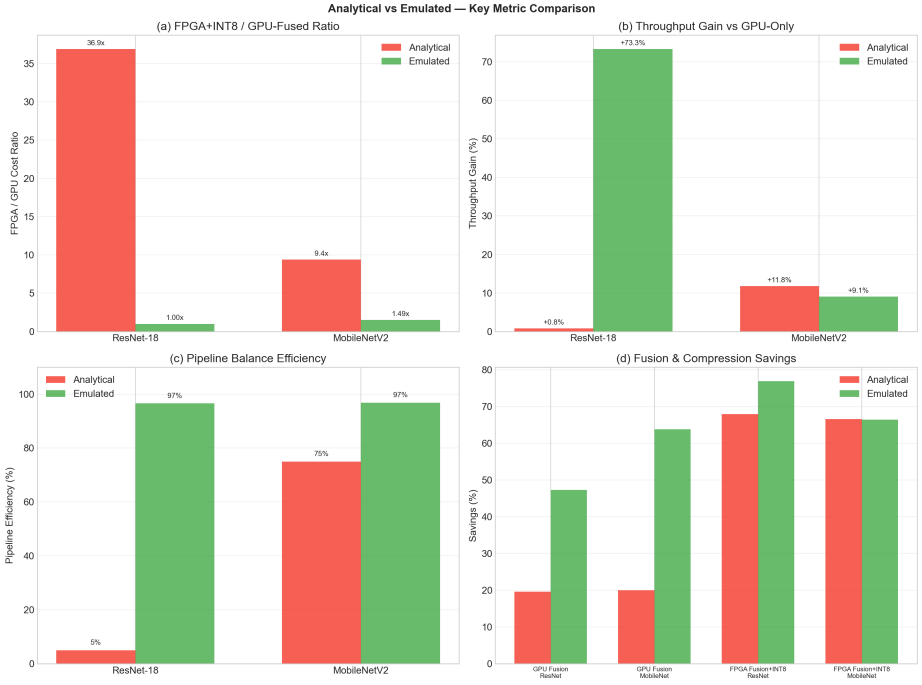


Fig. 4. Analytical vs emulated: (a) FPGA/GPU ratio, (b) throughput gain, (c) pipeline efficiency, (d) fusion savings.

(peak FLOPS) to 1.0–1.5× (memory-limited). Analytical models relying on peak FLOPS systematically overestimate the GPU's advantage.

Emulated fusion savings (47–64% GPU, 45–77% FPGA) far exceed the analytical model's 20%, because eliminating intermediate tensor I/O has outsized impact in the memory-bound regime.

Calibration against published benchmarks shows the current Zynq 7020 is too slow for positive gains, but break-even requires only 1.5–3× improvement. UltraScale+ at 5–8× delivers +35% to +140% gain.

FPGA calibration factors differ 3.2× between architectures (workload-specific implementations), while GPU varies less than 9%. The calibration predictor handles this with 0.20% regret when trained on diverse architectures.

Limitations include: efficiency factor assumptions rather than measured hardware behavior, no INT8 accuracy evaluation, sequential topology assumption, and the need for hardware validation.

Future work targets validation on Zynq UltraScale+, real per-layer timing, quantization accuracy evaluation, and extension to transformers.

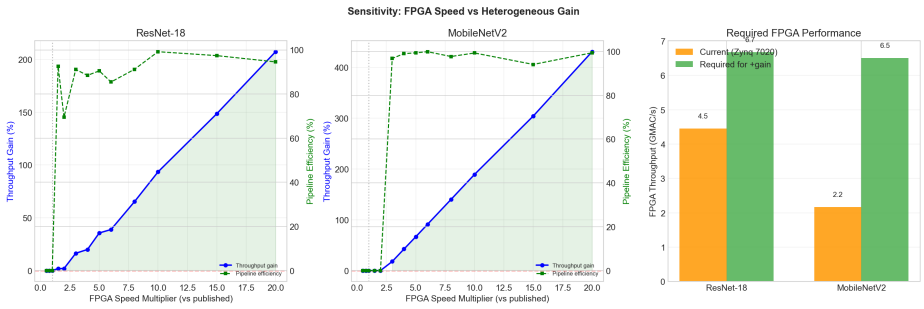


Fig. 5. FPGA speed sensitivity: throughput gain vs speed multiplier and required GMAC/s for break-even.

5 Conclusion

This paper presented an adaptive framework for heterogeneous GPU-FPGA CNN inference, evaluated through analytical simulation and functional emulation. The analytical model shows up to 11.8% gain for depthwise architectures with 0.20% predictor regret. The emulation reveals memory-bound execution, collapsing the FPGA/GPU ratio from 36.9× to 1.0×. Calibration shows break-even at 1.5–3× Zynq 7020. UltraScale+ projections show +35% to +140% throughput at above 90% pipeline efficiency. The framework adapts automatically from conservative tail splits to aggressive partitioning as FPGA performance scales.

A Supplementary Figures

A.1 Trial 1: Analytical Simulation



Fig. 6. Predictor scheduling regret: original (2 architectures) vs expanded (6 architectures) training. VGG-11 regret drops from 30% to near zero.

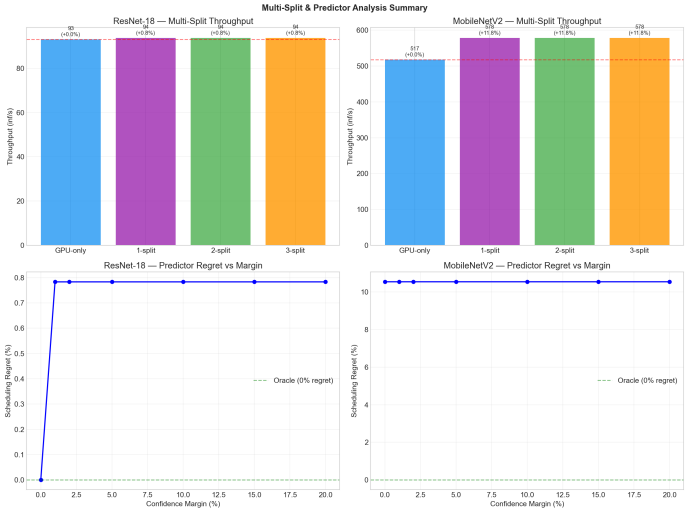


Fig. 7. Multi-split throughput comparison (top) and predictor regret vs confidence margin (bottom). Single-split is sufficient at current FPGA performance levels.

A.2 Trial 2: Functional Emulation

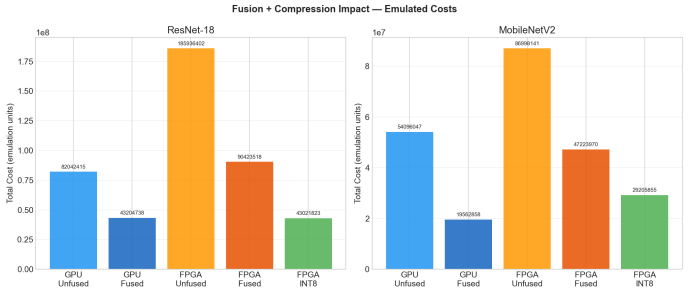


Fig. 8. Fusion and INT8 compression impact on emulated cost.

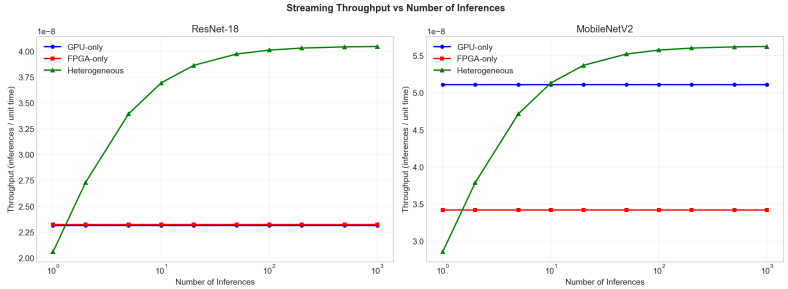


Fig. 9. Streaming throughput vs number of inferences.

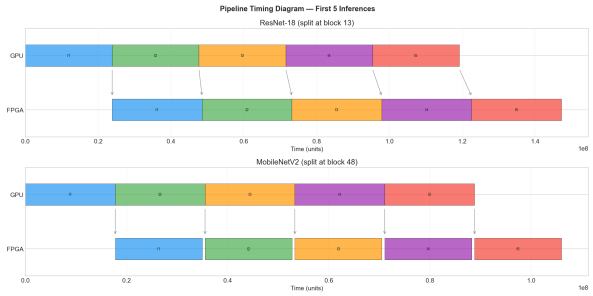


Fig. 10. Pipeline timing diagram showing overlapping GPU and FPGA execution for the first 5 inferences. Each color represents one input frame.

A.3 Calibrated Results

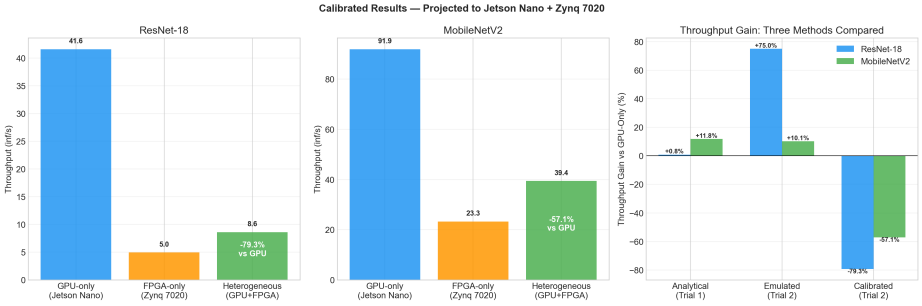


Fig. 11. Calibrated results: (a,b) projected throughput on Jetson Nano + Zynq 7020 target hardware, (c) throughput gain comparison across analytical, emulated, and calibrated modeling levels.

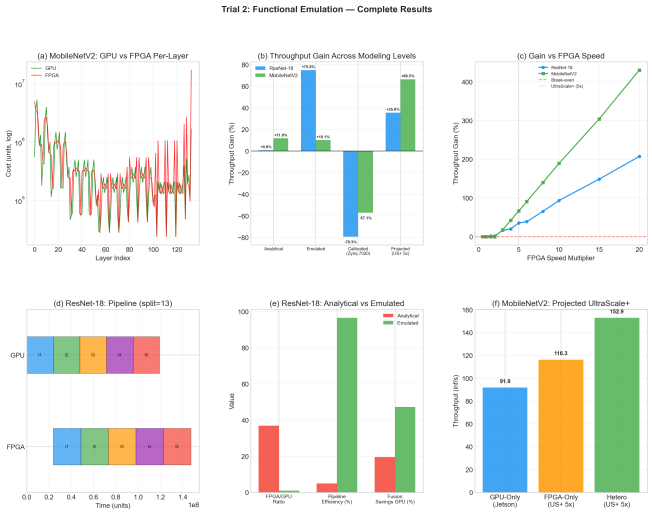


Fig. 12. Trial 2 complete summary: (a) per-layer GPU vs FPGA costs, (b) four-level throughput gain comparison, (c) gain vs FPGA speed, (d) pipeline timing, (e) analytical vs emulated metrics, (f) UltraScale+ projected throughput.